System Design

Author: Vladislav Stefanov

Contents

System Design **1**
- Document versions . . . 1
- Overview . . . 1
- Architecture . . . 2
- Input . . . 2
- Perception . . . 2
  - 2D detection . . . 2
  - 3D localization . . . 3
- State estimation . . . 3
  - Kalman filter design . . . 3
  - Noise parameters and configurations . . . 4
  - Per-frame cycle . . . 4
  - Coasting . . . 4
  - Initialization . . . 5
- Prediction . . . 5
- Output . . . 5
- Deployment . . . 5
- Design decisions . . . 6
- Known limitations . . . 6

List of Figures

1 Pipeline architecture: three stages (perception, state estimation and output) connected by data flow from camera input to annotated video and recorded trajectories. YOLO v8x and ZED stereo depth estimation are fused into a 3D position, which a Kalman filter uses to track position and velocity and extrapolate 0.2 s ahead. 2

Document versions

Version	Date	Description
1.0	2026-06-05	Initial version

Overview

The pipeline detects a moving object in a ZED stereo camera stream, tracks its 3D position and velocity over time using a Kalman filter, and predicts where the object will be 0.2 seconds ahead. It is designed for real-time operation at 30 fps and is packaged for deployment on mobile robotics platforms.

The system runs as a per-frame loop with three stages:

1. **Perception** - detect the object in 2D and compute its 3D position from the camera depth data
2. **State estimation** - smooth the noisy 3D measurements and estimate velocity using a Kalman filter
3. **Prediction** - extrapolate the current state 0.2 s forward

---

Architecture

The diagram below shows how data flows through the pipeline. The camera (or SVO2 recording) is the single input source. Each frame provides both an RGB image and an XYZ depth map to the perception stage. The Kalman filter receives one 3D measurement per frame, updates its state and produces a predicted trajectory. Both the raw measurements and predictions are written to CSV for evaluation. An annotated video is produced as a visual output.

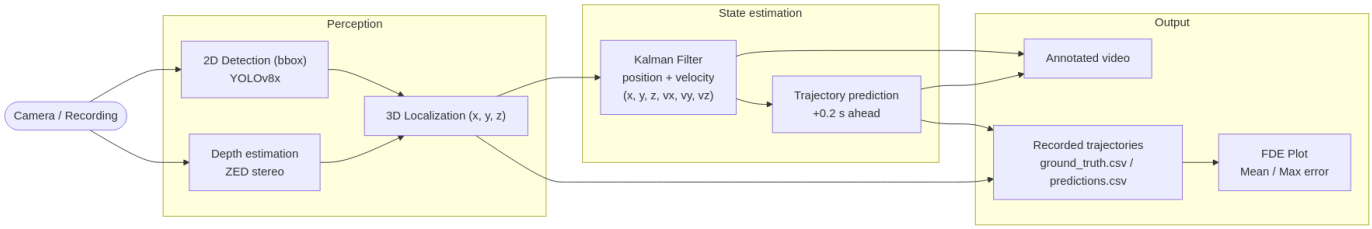


Figure 1: Pipeline architecture: three stages (perception, state estimation and output) connected by data flow from camera input to annotated video and recorded trajectories. YOLO v8x and ZED stereo depth estimation are fused into a 3D position, which a Kalman filter uses to track position and velocity and extrapolate 0.2 s ahead.

Input

Source: ZED stereo camera (live) or SVO2 recording file

The ZED SDK is opened in non-real-time mode for SVO2 playback, so frames are processed as fast as compute allows. Coordinate units are set to meters at open time, so all downstream code operates in a single consistent unit.

Each frame provides two pixel-aligned arrays from the SDK:

Array	Shape	Type	Content
`left_rgb`	(H, W, 3)	uint8	Left camera RGB image
`xyz_map`	(H, W, 4)	float32	Per-pixel (x, y, z) in meters, camera frame

The XYZ map is pre-computed by the ZED SDK using NEURAL depth mode. Index (v, u) in the XYZ map and index (v, u) in the RGB image refer to the same physical point in the scene. Pixels where depth computation failed contain NaN or inf values.

The camera coordinate frame is:

- X: lateral (positive = right)
- Y: vertical (positive = down)
- Z: depth from camera (distance from lens)

Perception

2D detection

YOLO v8x (`yolov8x.pt`) runs on the left RGB image. Detection is filtered to COCO class 32 (sports ball) with a confidence threshold of 0.15. If multiple detections pass the threshold, the highest-confidence box is selected.

The output is a bounding box `(x1, y1, x2, y2)` in pixel coordinates, or no detection if no qualifying box was found.

The confidence threshold of 0.15 is lower than the YOLO default (0.25) to reduce missed detections. False positives are tolerated because the Kalman filter is robust to single-frame noise and predictions are only issued after at least two consecutive corrections.

3D localization

The 3D position is computed from the bounding box using front-face depth sampling. The algorithm runs as follows:

1. Extract the XYZ map patch for all pixels within the bounding box.
2. Identify all pixels with a finite, positive Z value. If fewer than 5 valid pixels exist, treat the frame as a miss.
3. Find the minimum Z among the valid pixels - this is the depth of the object's front surface.
4. Select only pixels within 0.5 cm of the minimum Z. This is the front-face cluster.
5. Compute the pixel centroid (u, v) of the front-face cluster by averaging the pixel coordinates.
6. Look up the 3D point at pixel (u, v) in the XYZ map to get X and Y.
7. Use the median Z of the front-face cluster as the final depth.

The result is a (X, Y, Z) position in meters in the camera frame.

Why front-face sampling over the bounding box center: the prototype used a simple XYZ lookup at the bbox center pixel, which produced large Y errors because floor and background pixels near the bottom of the box pulled the centroid toward an incorrect Y position. Front-face sampling selects only the closest pixels, which belong to the object's front surface and removes most background interference without requiring segmentation. This is the primary reason the current pipeline achieves lower position error than the prototype.

Why median for depth: the median is more robust than the mean for the front-face cluster depth. Pixels at object edges can have anomalous depth values due to stereo occlusion between the two camera lenses. The median is less affected by these outliers than the mean would be.

State estimation

Kalman filter design

A linear constant-velocity Kalman filter tracks the object in 3D camera space. The design uses six state variables and three measurements.

State vector:

$$\mathbf{x} = [x,\ y,\ z,\ v_x,\ v_y,\ v_z]^T$$

Position (x, y, z) is observed each frame via the depth map. Velocity (vx, vy, vz) is never directly measured - it is inferred by the filter over time from the sequence of position measurements.

Measurement vector (position only):

$$\mathbf{z} = [x,\ y,\ z]^T$$

Transition matrix (constant-velocity model):

$$F = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \Delta t = \frac{1}{30}\ \text{s}$$

At each frame, position advances by velocity times the time step. Velocity is carried forward unchanged - this is the constant-velocity assumption.

Observation matrix (extracts position rows from the 6-variable state):

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Process noise matrix (diagonal, values depend on configuration):

$$Q = \text{diag}([0.0001,\ 0.0001,\ 0.0001,\ q_v,\ q_v,\ q_v])$$

Position process noise is small (0.0001 m²) because position changes predictably via the motion model between frames. Velocity process noise q_v is larger and configurable to control how quickly the filter allows velocity to change.

Measurement noise matrix (diagonal, values depend on configuration):

$$R = \text{diag}([r_x,\ r_y,\ r_z])$$

Values in R encode how much the filter trusts each camera measurement. Higher values cause the filter to rely more on the motion model and smooth out measurement noise more aggressively.

Initial covariance: the 6x6 identity matrix. This expresses moderate uniform uncertainty about all state variables at the moment of first detection and causes the filter to converge quickly toward the first few measurements.

A full derivation of the predict-correct equations and the Kalman gain is in the Kalman Filter document.

Noise parameters and configurations

Four configurations were evaluated, varying the velocity process noise and measurement noise independently:

Configuration	vel process noise (q_v)	r_x (m²)	r_y (m²)	r_z (m²)
baseline	0.01	0.001	0.001	0.001
zed_noise	0.01	0.000064	0.000121	0.000064
high_vel	0.1	0.001	0.001	0.001
zed_noise_high_vel	0.1	0.000064	0.000121	0.000064

The `zed_noise` measurement values are derived from the static ZED accuracy experiment (standard deviation: X = 0.8 cm, Y = 1.1 cm, Z = 0.8 cm, converted to variance in m²). The baseline uses 0.001 m² per axis, which corresponds to approximately 3.2 cm standard deviation - higher than the actual ZED accuracy, so the baseline trusts the model more than necessary.

The `high_vel` process noise of 0.1 allows velocity to adapt faster to acceleration. The `zed_noise_high_vel` configuration combines both changes. Evaluation results show all three non-baseline configurations achieve equivalent prediction accuracy, each slightly better than baseline. Details are in the Project Report and sweep_results.csv.

Per-frame cycle

Each frame runs in the following order:

1. If the filter is initialized: run the **predict** step to advance the state by one frame using F.
2. Run YOLO detection and front-face depth sampling to obtain a 3D measurement.
3. If a valid measurement was obtained: run the **correct** step to blend the prediction with the measurement. Increment the corrections counter. Reset the missed counter to zero.
4. If no valid measurement: increment the missed counter.
5. If corrections >= 2 and missed < `MAX_MISSED`: compute and store the 6-step prediction sequence.

The corrections threshold of 2 ensures the velocity estimate has had at least one measurement-to-measurement interval before predictions are issued.

Coasting

If detection fails for up to `MAX_MISSED = 5` consecutive frames, the filter continues on the predict step only. The state advances using the constant-velocity motion model without any correction. Predictions are still issued during coasting. If 5 frames pass without a detection, prediction output is suppressed until the object is re-detected and the filter is re-initialized.

Initialization

The filter is initialized on the first valid 3D detection. The state is set to the measured position with zero velocity:

$$\mathbf{x}_0 = [x_{\text{meas}},\ y_{\text{meas}},\ z_{\text{meas}},\ 0,\ 0,\ 0]^T$$

Zero initial velocity is assumed because there is no information to estimate it from a single detection.

Prediction

Forward prediction is computed by linear extrapolation from the current filter state after the correct step:

$$\hat{p}(h) = \begin{bmatrix} x + v_x h \\ y + v_y h \\ z + v_z h \end{bmatrix}$$

where (x, y, z) and (v_x, v_y, v_z) are taken from the current filter state, and h is the horizon in seconds. Six predictions are computed per frame at $h = 1/30, 2/30, \ldots, 6/30$ seconds. The final step at $h = 0.2$ s is the primary output.

Why linear extrapolation and not repeated predict steps: running the KF predict step forward 6 times would also advance the covariance matrix, corrupting the filter's uncertainty estimate for subsequent measurements. Linear extrapolation gives an identical result under the constant-velocity model while leaving the filter state unchanged.

Limitation: the extrapolation does not propagate uncertainty. The predicted position has no attached confidence interval. Uncertainty grows with the horizon, but this is not reflected in the output.

Output

The pipeline produces three outputs per evaluation run.

Annotated video (MP4, 1920x1080, 30 fps)

Overlay	Color	Meaning
Tracked position trail	Green	Recent smoothed positions
Current position circle	Green	Valid detection this frame
Current position circle	Orange	Coasting - no detection, Kalman predicted position used
Predicted position trail	Red	Recent 0.2 s ahead predictions
Predicted position circle	Red	Predicted position 0.2 s ahead this frame
Position labels	-	Current x, y, z values and predicted x, y, z with `+0.2s` tag

ground_truth.csv - one row per detected frame: frame index and raw (x, y, z) position from the front-face sampling step. This is the pipeline's own localization output, not an independent ground truth.

predictions.csv - one row per frame with a valid prediction: frame index and the full 6-step predicted position sequence.

These two files are read by the evaluation script to compute ADE, FDE, latency, throughput and RAM usage per configuration. See sweep_results.csv for aggregated results.

Deployment

The pipeline is packaged as a Docker container. The container includes all Python dependencies: ZED SDK, Ultralytics YOLO, OpenCV, NumPy, psutil and Matplotlib. SVO2 dataset paths and output paths are passed in at runtime. This allows the pipeline to be deployed on the target robotics platforms (Jackal UGV, Avular Origin One, Triangular Robot Platform) without environment setup on the host system.

Design decisions

YOLO v8x over smaller YOLO variants: v8x is the most accurate YOLOv8 model. Detection rate is more critical than throughput for this pipeline, provided the system runs at or above 30 fps. The evaluation showed 35+ fps, confirming v8x leaves sufficient headroom on the test machine.

Front-face depth sampling over bbox center lookup: the prototype used a direct XYZ lookup at the bounding box center pixel. This caused systematic Y errors because floor and background pixels near the bottom of the bbox shifted the apparent position downward. Front-face sampling isolates the object's closest surface and resolves this without segmentation. The static accuracy experiment confirmed a remaining Y bias of -5.0 cm (camera above ball geometry) and Z bias of +4.1 cm (ZED NEURAL depth at close range), both of which are stable and correctable.

Constant-velocity Kalman filter over higher-order models: constant velocity is the simplest model that provides both smoothing and velocity estimation from position-only measurements, with no training data or tuning beyond two scalar noise parameters. For short-horizon prediction of smooth trajectories (straight and diagonal throws) it achieves 1.3 cm mean ADE at 0.2 s. A constant-acceleration or ballistic model would improve accuracy for bounces and gravity arcs at the cost of additional parameters.

Linear extrapolation over repeated KF predict steps: under a linear constant-velocity model both approaches produce the same predicted position. Linear extrapolation is used to avoid advancing the covariance matrix and corrupting the filter's uncertainty estimate.

Calibrated measurement noise over arbitrary defaults: setting the measurement noise from the ZED static accuracy experiment (0.000064-0.000121 m^2 per axis) rather than the baseline default (0.001 m^2 per axis) consistently reduces ADE and FDE. The baseline default over-trusts the motion model because it makes the filter believe the camera is noisier than it actually is.

Known limitations

Single-object tracking: the pipeline assumes one object is present at a time. Extending to multi-object tracking requires detection-to-track data association across frames and one independent filter instance per tracked object.

Constant-velocity model: the filter does not model acceleration, gravity or bouncing. Curved trajectories or sudden velocity changes cause prediction lag. Thrown or rolling objects on smooth trajectories are handled well, but bounces and rapid direction changes are not.

Ground truth from the same pipeline: the evaluation ground truth is produced by the same YOLO and ZED pipeline that drives predictions. Systematic errors in detection or depth cancel between the prediction and the future ground truth frame. ADE and FDE likely underestimate true prediction error relative to real-world positions. An evaluation against independently annotated ground truth would give a more accurate picture.

Limited accuracy characterization coverage: the ZED accuracy experiment was conducted at one height (Y = 0.28 m, floor level) and depths up to 0.77 m. Y and Z biases may differ at other heights or longer distances.

No uncertainty on forward prediction: the predicted position has no associated confidence interval. Uncertainty grows with the prediction horizon but is not propagated or reported.